K J Somaiya School of Engineering
Somaiya Vidyavihar University, Vidyavihar (E), Mumbai – 400077

Department of Electronics and Computer Science

Mini Project 1

Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions

Based on: N. Halko, P. G. Martinsson, J. A. Tropp, arXiv:0909.4061v2, SIAM Review 53(2):217–288, 2011.

Name	Roll No.	Responsibility
Naman Lodha	16015024039	SRFT, ID, CUR Decompositions
Aditya Lokhande	16015024040	Experiments, Visualizations
Aryan Lomte	16015024041	Core Randomized SVD Algorithms

Branch: Computer Science and Business Systems (CSBS)
Division: A **Batch:** A2
Subject: Linear Algebra and Optimization (LAO)
AY: 2025–26
Date: March 19, 2026

Contents

1 Project Title **2**

2 Project Description **2**
2.1 Background and Motivation 2
2.2 The Randomized Solution 2
2.3 Our Implementation Scope 3

3 Concept Used **3**
3.1 Primary Mathematical Concepts 3
3.2 Key Mathematical Framework 4

4 Application Area **4**

5 Expected Outcome **5**
5.1 Implementation Deliverables 5
5.2 Expected Experimental Results 5

6 Dataset Used **6**
6.1 Primary Datasets (Synthetic, Generated In-Code) 6
6.2 Optional Extension Dataset 6

7 Tools and Libraries **6**

8 Tentative Algorithm / Methodology **6**
8.1 Stage A: Randomized Range Finder (Algorithm 4.4 — Stabilized) . . . 6
8.2 Stage A: Algorithm 4.1 — Basic Range Finder 7
8.3 Stage A: Algorithm 4.2 — Adaptive Range Finder 7
8.4 Stage A: Algorithm 4.5 — SRFT Fast Range Finder 8
8.5 Stage B: Algorithm 5.1 — Direct SVD 8
8.6 Interpolative Decomposition (Section 5.2) 8
8.7 CUR Decomposition (Section 2.1.4) 8

9 Team Members with Name & Roll Numbers **9**

10 Division / Batch **9**

11 Bug Fixes Incorporated **9**

12 References **10**

1 Project Title

Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions

Reference Paper: N. Halko, P. G. Martinsson, J. A. Tropp, "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions," *SIAM Review* 53(2):217–288, 2011. (URL: arxiv.org/abs/0909.4061)

2 Project Description

2.1 Background and Motivation

Matrix decompositions — particularly the Singular Value Decomposition (SVD), QR factorization, and eigenvalue decomposition — are the foundational building blocks of modern data science, machine learning, and scientific computing. They underpin algorithms for Principal Component Analysis (PCA), dimensionality reduction, latent semantic analysis, recommendation engines, image compression, and numerical solutions to partial differential equations.

However, all classical algorithms for these decompositions share a fundamental bottleneck: the SVD of an $m \times n$ matrix requires $\mathcal{O}(mn \min(m, n))$ floating-point operations. For matrices arising in genomics (millions of genes $\times$ thousands of samples), text mining (vocabulary size $\times$ document count), and image databases, this cost is simply prohibitive. Three structural challenges of modern data motivate a new approach:

1. **Computational cost**: Classical algorithms are not well-adapted to modern parallel architectures such as GPUs and distributed clusters, where memory bandwidth is the bottleneck, not raw FLOPS.

2. **Memory access**: For matrices too large to fit in RAM (out-of-core), classical methods require $\mathcal{O}(k)$ passes over the data. Even for modest k, this can mean hours of disk I/O.

3. **Data imprecision**: Classical SVD computes machine-precision results, which is wasteful when the input data is inherently noisy or only approximately known. Trading some accuracy for speed is both mathematically sound and practically desirable.

2.2 The Randomized Solution

The key insight of Halko, Martinsson, and Tropp (2009) is powerful in its simplicity: for matrices with *numerical low rank* — i.e., matrices whose singular values decay rapidly to zero — a *random* projection of the matrix captures nearly all of its important structure. Concretely:

- Draw a random test matrix $\Omega \in \mathbb{R}^{n \times l}$ (Gaussian or structured).
- Compute the sample matrix $Y = A\Omega \in \mathbb{R}^{m \times l}$.
- The columns of Y are random samples from the range (column space) of A.
- Orthonormalize Y to get Q. Then $A \approx QQ^\top A$ with high probability.

This is not mere heuristic intuition — the paper provides *rigorous probabilistic error bounds* (Theorems 1.1 and 1.2) showing the approximation is optimal up to a small factor with probability exponentially close to 1. The method has since been adopted in production libraries: `sklearn.utils.extmath.randomized_svd`, `torch.linalg.svd_lowrank`, and Facebook's FBPCA library.

2.3 Our Implementation Scope

This project implements the complete randomized framework from the paper, covering:

- All four Stage A algorithms (Algorithms 4.1, 4.2, 4.3/4.4, 4.5)
- Stage B direct SVD (Algorithm 5.1)
- Interpolative Decomposition (Section 5.2)
- CUR Decomposition (Section 2.1.4)
- Empirical verification of Theorems 1.1 and 1.2
- Five comprehensive numerical experiments with visualizations

3 Concept Used

3.1 Primary Mathematical Concepts

1. **Randomized Singular Value Decomposition (Randomized SVD)**
   The core algorithm. Replaces the $\mathcal{O}(mn^2)$ classical SVD with a two-stage randomized procedure costing $\mathcal{O}(mnk)$ or $\mathcal{O}(mn \log k)$.

2. **QR Factorization**
   Every $m \times n$ matrix A (with $m \geq n$) can be written as $A = QR$ where $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R \in \mathbb{R}^{n \times n}$ is upper triangular. Used in every stage of the algorithm to orthonormalize sample matrices.

3. **Power / Subspace Iteration**
   To handle matrices with slowly-decaying singular spectra, we apply the range finder to $(AA^\top)^q A$ instead of A. Since $\sigma_j((AA^\top)^q A) = \sigma_j(A)^{2q+1}$, the effective spectrum decays exponentially faster. Just $q = 1$ or $q = 2$ is sufficient in practice.

4. **Subsampled Randomized Fourier Transform (SRFT)**
   A structured random matrix of the form $\Omega = \sqrt{n/l}\, R\, F\, D$ where D applies random ± 1 sign flips, F is the DFT matrix applied via FFT, and R subsamples l rows. Reduces Stage A cost from $\mathcal{O}(mnl)$ to $\mathcal{O}(mn \log l)$.

5. **Interpolative Decomposition (ID)**
   Represents $A \approx A_{:,J}\, X$ using k *actual columns* of A. The factor X satisfies $|X_{ij}| \leq 2$ (bounded entries) and $X_{J,:} = I_k$ (columns of $A_{:,J}$ are reproduced exactly). More interpretable than SVD.

6. **CUR Decomposition**
   $A \approx C\, U\, R$ where $C = A_{:,J}$ is k columns, $R = A_{I,:}$ is k rows, and $U = W^\dagger$

(pseudoinverse of the $k \times k$ intersection submatrix $W = A_{I,J}$). Entirely in the original data space.

7. **Johnson–Lindenstrauss Lemma**
   A random linear map $f : \mathbb{R}^n \to \mathbb{R}^l$ approximately preserves pairwise distances when $l = \mathcal{O}(\varepsilon^{-2} \log n)$. This underpins why random test matrices capture the action of A.

8. **Measure Concentration (Gordon's Theorem)**
   For a Gaussian random matrix Ω, the approximation error $\|A - QQ^\top A\|$ concentrates sharply around its expectation. The actual error is almost surely very close to the average case, making the expected-error bounds practically meaningful.

9. **Principal Component Analysis (PCA)**
   An application. Given a centered data matrix X, PCA computes the k directions of maximum variance via the leading k right singular vectors. Randomized SVD computes these at 3–11× lower cost with negligible accuracy loss.

3.2 Key Mathematical Framework

The two-stage decomposition framework:

$$\underbrace{A}_{m\times n} \approx \underbrace{Q}_{m\times l}\,\underbrace{Q^\top A}_{l\times n}, \qquad Q^\top Q = I_l, \quad l = k + p,$$

where k is the target rank and $p \geq 0$ is the oversampling parameter.

Theorem 1.1 (Basic error bound): With Gaussian Ω and $p \geq 2$:

$$\mathbb{E}\,\|A - QQ^\top A\|_2 \;\leq\; \left(1 + \sqrt{\frac{k}{p-1}}\right)\sigma_{k+1} \;+\; \frac{e\sqrt{k+p}}{p}\sqrt{\sum_{j>k}\sigma_j^2}.$$

Theorem 1.2 (Power iteration bound): With q power iteration steps:

$$\mathbb{E}\,\|A - U_k\Sigma_k V_k^\top\|_2 \;\leq\; \left[1 + 4\sqrt{\frac{2\min(m,n)}{k+1}}\right]^{\frac{1}{2q+1}} \sigma_{k+1}.$$

As q increases, the bracket converges to $1^{1/(2q+1)} \to 1$ exponentially.

4 Application Area

1. **Image Compression**
   A grayscale image is an $m \times n$ matrix. A rank-k randomized SVD approximation stores only $k(m+n)$ floats instead of mn, achieving compression ratio $k(m+n)/(mn)$. At $k = 50$ on a 256×256 image, this is 39% storage with $< 9\%$ reconstruction error.

2. **Dimensionality Reduction / PCA**
   Given a high-dimensional data matrix $X \in \mathbb{R}^{d\times N}$ (e.g., $d = 50000$ gene expressions for $N = 500$ patients), classical PCA costs $\mathcal{O}(d^2 N)$. Randomized PCA for the top $k = 50$ components reduces this to $\mathcal{O}(dNk)$, enabling analysis previously considered computationally infeasible.

3. **Recommendation Systems**
User–item interaction matrices (Netflix: 10^5 users $\times$ 10^4 movies) are low-rank in practice. Randomized SVD finds the latent factors faster, enabling real-time retraining as new ratings arrive.

4. **Large-Scale Numerical Linear Algebra**
For matrices stored on disk or arriving as a stream, randomized algorithms need only $2q+1$ passes over the data (vs. $\mathcal{O}(k)$ for Krylov methods), making them the only feasible option for truly massive datasets.

5 Expected Outcome

5.1 Implementation Deliverables

Three Python modules will be submitted:

File	Owner	Algorithms
`randomized_svd_core.py`	Aryan Lomte (041)	Alg. 4.1, 4.2, 4.4, 5.1, error estimat
`srft_and_decompositions.py`	Naman Lodha (039)	Alg. 4.5 (SRFT), ID (Sec. 5.2), CU
`experiments_and_applications.py`	Aditya Lokhande (040)	Experiments 1–5, all plots and figu

5.2 Expected Experimental Results

1. **Correctness of Randomized SVD**
Randomized SVD achieves $\|A - \tilde{A}\|_F / \|A\|_F \approx 10^{-3}$ matching `scipy.linalg.svd` accuracy on well-conditioned matrices. The difference in the top-k singular values will be $< 0.1\%$.

2. **Computational Speedup**
Expected $3\times$ speedup at $m = 400$, growing to 10–11$\times$ at $m = 1000$, since classical SVD scales as $\mathcal{O}(m^3)$ while randomized SVD scales as $\mathcal{O}(m^2 k)$.

3. **Theorem 1.1 Verification (Oversampling)**
For a 500×400 matrix with $\sigma_j = 1/j$ decay, fixing $k = 20$, $q = 2$, and sweeping $p \in [0, 50]$: error converges from $\approx 5\sigma_{k+1}$ at $p = 0$ to $\approx 1.05\sigma_{k+1}$ at $p = 10$. Standard deviation across trials collapses to near-zero by $p = 15$.

4. **Theorem 1.2 Verification (Power Iteration)**
On a flat-spectrum matrix ($\sigma_j = 1/\sqrt{j}$), $q = 0$ gives error $\approx 7.7\times$ larger than σ_{k+1}. $q = 1$ reduces this to $\approx 1.7\times$, and $q = 2$ reduces it to $\approx 1.1\times$ — essentially optimal. Fast-decay matrices show negligible difference across q values.

5. **SRFT vs. Gaussian**
At $m = 2000$, $n = 1500$, $l = 60$: SRFT achieves the same relative error ($\approx 3.3 \times 10^{-3}$) as Gaussian while reducing Stage A wall-time by $\approx 1.5\times$ due to FFT speedup.

6. **Image Compression Trade-off Curve**
At ranks $k \in \{5, 20, 50, 100\}$: compression ratios of $\{3.9\%, 15.6\%, 39.1\%, 78.1\%\}$ with relative errors $\{42.3\%, 18.1\%, 8.9\%, 4.1\%\}$ respectively.

6 Dataset Used

6.1 Primary Datasets (Synthetic, Generated In-Code)

All core experiments use programmatically generated synthetic matrices, requiring no external downloads. This allows precise control over all experimental parameters.

Matrix Type	Construction	Purpose
Low-rank + noise	$A = U\Sigma V^\top + \eta E$	Core benchmarking
Fast-decay spectrum	$\sigma_j = j^{-5}$	Theorem 1.2, power iteration
Moderate-decay spectrum	$\sigma_j = j^{-2}$	General experiments
Flat spectrum	$\sigma_j = j^{-0.5}$	Worst-case analysis
Synthetic image	Outer-product + noise, 256×256	Compression demo

Construction formula:

$$A = U \operatorname{diag}(\sigma_1, \ldots, \sigma_r)\, V^\top + \eta E, \quad \sigma_j = j^{-\alpha}, \quad E_{ij} \sim \mathcal{N}(0,1).$$

Here U, V are random orthonormal matrices, α controls the spectral decay rate, and $\eta = 0.01$ controls noise level.

6.2 Optional Extension Dataset

Dataset	Source	Size
MNIST Handwritten Digits	Kaggle (URL: kaggle.com/datasets/hojjatk/mnist-dataset)	60000 ×

7 Tools and Libraries

Library	Version	Purpose
Python	3.12	Primary implementation language
NumPy	1.26	Matrix operations, random generation, core linear algebra
SciPy (linalg)	1.12	QR decomposition with pivoting, SVD, eigendecomposition
SciPy (fft)	1.12	Fast Fourier Transform for SRFT Algorithm 4.5
Matplotlib	3.8	All visualizations, benchmark plots, dashboard figures
time / timeit	stdlib	Accurate wall-clock benchmarking

No deep learning frameworks are required. The entire implementation uses only the NumPy/SciPy ecosystem, ensuring reproducibility and portability.

8 Tentative Algorithm / Methodology

8.1 Stage A: Randomized Range Finder (Algorithm 4.4 — Stabilized)

Input: $A \in \mathbb{R}^{m\times n}$, target rank k, oversampling p, power iteration count q.

1. Set $l = k + p$ (total sample count).
2. Draw random Gaussian test matrix: $\Omega \in \mathbb{R}^{n\times l}$, $\Omega_{ij} \sim \mathcal{N}(0,1)$ i.i.d.
3. Compute initial sample matrix: $Y = A\Omega \in \mathbb{R}^{m\times l}$.

4. **Power iteration** (q steps, with QR stabilization at each step):
   (a) Compute QR of Y: $(Q_{\text{temp}}, _) = \text{QR}(Y)$
   (b) Form $Z = A^\top Q_{\text{temp}}$ and compute QR: $(\tilde{Q}, _) = \text{QR}(Z)$
   (c) Update: $Y = A\tilde{Q}$
5. Final orthonormalization: $(Q, R) = \text{QR}(Y)$. Return $Q \in \mathbb{R}^{m\times l}$.

Why QR stabilization matters: Without intermediate QR steps, floating-point errors in repeated multiplication by A and $A^\top$ cause numerical collapse — information about small singular modes is lost. The QR steps cost $\mathcal{O}(l^2 m)$ extra (negligible) but preserve full precision.

Python implementation:

```
def randomized_range_finder_power(A, l, q=2, random_state=None):
    rng = np.random.default_rng(random_state)
    Omega = rng.standard_normal((A.shape[1], l))
    Y = A @ Omega
    for _ in range(q):
        Q_temp, _ = qr(Y, mode='economic')       # Stabilize
        Z = A.T @ Q_temp
        Q_temp2, _ = qr(Z, mode='economic')      # Stabilize
        Y = A @ Q_temp2
    Q, _ = qr(Y, mode='economic')
    return Q
```

8.2 Stage A: Algorithm 4.1 — Basic Range Finder

When $q = 0$ (fast spectrum or large oversampling sufficient):

1. Draw $\Omega \in \mathbb{R}^{n\times l}$ with i.i.d. $\mathcal{N}(0, 1)$ entries.
2. $Y = A\Omega$.
3. $(Q, _) = \text{QR}(Y)$. Return Q.

Cost: $\mathcal{O}(mnl + ml^2)$. When $l \ll n$, dominated by matrix multiply.

8.3 Stage A: Algorithm 4.2 — Adaptive Range Finder

For the fixed-precision problem (numerical rank unknown):

1. Initialize: draw $r = 10$ Gaussian vectors; form $Y = A\Omega \in \mathbb{R}^{m\times r}$; $Q = []$.
2. **While** $\max_i \|(I - QQ^\top)y_i\| > 10\sqrt{2/\pi}\,\varepsilon$:
   (a) Select the column y_j with largest residual norm.
   (b) Orthogonalize against current Q: $q_{\text{new}} = (I - QQ^\top)y_j / \|(I - QQ^\top)y_j\|$.
   (c) Append q_{new} to Q.
   (d) Draw a new Gaussian sample, project out current basis.
3. Return Q.

8.4 Stage A: Algorithm 4.5 — SRFT Fast Range Finder

For large dense matrices where $\mathcal{O}(mn \log l)$ cost is needed:

1. Draw $d_1, \ldots, d_n \sim \text{Uniform}(\{-1, +1\})$; form AD by scaling columns.
2. Apply FFT column-wise: $\hat{A} = \mathcal{F}(AD)/\sqrt{n}$.
3. Sample l column indices I uniformly without replacement from $\{1, \ldots, n\}$.
4. $Y = \sqrt{n/l} \cdot \text{Re}(\hat{A}_{:,I})$.
5. $(Q, _) = \text{QR}(Y)$. Return Q.

Python implementation:

```
def srft_range_finder(A, l):
    m, n = A.shape
    rng = np.random.default_rng(42)
    signs = rng.choice([-1.0, 1.0], size=n)
    AD = A * signs[np.newaxis, :]              # Apply D
    AF_complex = fft(AD, axis=1) / np.sqrt(n) # Apply F via FFT
    idx = rng.choice(n, size=l, replace=False) # Subsample R
    Y = np.sqrt(n / l) * np.real(AF_complex[:, idx])
    Q, _ = qr(Y, mode='economic')
    return Q
```

8.5 Stage B: Algorithm 5.1 — Direct SVD

Given orthonormal Q from Stage A:

1. Form $B = Q^\top A \in \mathbb{R}^{l \times n}$.
2. Compute thin SVD: $B = \hat{U} \Sigma V^\top$.
3. Lift back to full space: $U = Q\hat{U}$.
4. Return truncated factors: $U_k, \Sigma_k, V_k^\top$.

8.6 Interpolative Decomposition (Section 5.2)

Represents A using actual columns of A:

1. Run Stage A to get Q.
2. Form projected matrix: $B = Q^\top A \in \mathbb{R}^{l \times n}$.
3. Apply **pivoted QR** to B: $B \cdot P = \hat{Q}\hat{R}$; take first k pivot indices J.
4. Selected columns: $C = A_{:,J}$.
5. Solve least-squares: $\min_X \|A - CX\|_F$. Return J, X, C.

8.7 CUR Decomposition (Section 2.1.4)

1. Compute column sampling probabilities: $p_j^{\text{col}} = \|A_{:,j}\|_2^2 / \|A\|_F^2$ (leverage scores).
2. Sample k columns: col_idx $\sim \text{Categorical}(p^{\text{col}})$.

3. Compute row sampling probabilities: $p_i^{\text{row}} = \|A_{i,:}\|_2^2 / \|A\|_F^2$.
4. Sample k rows: row_idx $\sim$ Categorical(p^{row}).
5. $C = A_{:,\text{col_idx}}, \quad R = A_{\text{row_idx},:}$.
6. $U = (A_{\text{row_idx},\text{col_idx}})^{\dagger}$. Return C, U, R.

9 Team Members with Name & Roll Numbers

Name	Roll No.	File	Detailed Responsibility
Naman Lodha	16015024039	`srft_and_decompositions.py`	Algorithm 4.5 (SRFT finder with FFT), Interp Decomposition via pivot (Sec. 5.2), CUR decomp with leverage-score sa (Sec. 2.1.4), Gaussian vs. benchmark
Aditya Lokhande	16015024040	`experiments_and_applications.py`	All 5 experiments, result error analysis, matplotlib izations, final 6-panel ben dashboard, imports from and Naman's modules
Aryan Lomte	16015024041	`randomized_svd_core.py`	Algorithms 4.1, 4.2, 4.4, 5. estimator, pipeline

10 Division / Batch

Field	Detail
Institution	K J Somaiya School of Engineering
University	Somaiya Vidyavihar University
Department	Electronics and Computer Science
Branch	Computer Science and Business Systems (CSBS)
Division	A
Batch	A2
Subject	Linear Algebra and Optimization (LAO)
Academic Year	2025–26
Submission Date	March 19, 2026

11 Bug Fixes Incorporated

During development, two correctness issues were identified and fixed:

1. **Adaptive Range Finder — Pool Memory Leak**
   Fix: Added a pool size cap of `MAX_POOL = r + max_rank`. bounded memory to $\mathcal{O}(m \cdot \text{max_rank})$.

2. **SRFT Range Finder — FFT Column Selection Order**

Fix: Column selection (`AF_complex[:, idx]`) is now performed on the full complex FFT output, and `np.real()` is applied *after* selection.

12 References

1. N. Halko, P. G. Martinsson, J. A. Tropp. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions." *SIAM Review*, 53(2):217–288, 2011. (URL: arxiv.org/abs/0909.4061)

2. G. H. Golub, C. F. Van Loan. *Matrix Computations*, 4th ed. 2013.

3. F. Pedregosa et al. "Scikit-learn: Machine Learning in Python." *JMLR*, 12:2825–2830, 2011.

Submitted to: LAO Lab Faculty Members, K J Somaiya School of Engineering.